Sometimes you might want to load a bean into the Spring context only under specific conditions. For example, you might want to define a bean responsible for generating test data only when you're working locally or testing the application. Profiles are logical groups of beans that are loaded into the Spring context only if the specified profile is active. Spring Boot also extends this concept to property files, allowing you to define groups of configuration data that are loaded only if a specific profile is active.

You can activate zero, one, or more profiles at a time. All beans not assigned to a profile will always be activated. Beans assigned to the `default` profile are only activated when no other profile is active.

This section presents Spring profiles in the context of two different use cases: feature flags and configuration groups.

## *USING PROFILES AS FEATURE FLAGS*

The first use case for profiles is for loading groups of beans only if a specified profile is active. The deployment environment shouldn't influence the reasoning behind the groupings too much. A common mistake is using profiles like `dev` or `prod` to load beans conditionally. If you do that, the application will be coupled to the environment, which is usually not what we want for a cloud native application.

Consider the case where you deploy applications to three different environments (development, test, and production) and define three profiles to load certain beans conditionally (`dev`, `test`, and `prod`). At some point you decide to add a staging environment, where you also want to enable the beans marked with the `prod` profile. What do you do? You have two options. Either you activate the `prod` profile in the staging environment (which doesn't make much sense), or you update the source